EXAMINATION QUESTION DIFFICULTY AND PERFORMANCE ANALYSIS

Project Report

1. Problem Statement

Develop a Python program that compares student answer strings with a predefined answer key, calculates question-wise success rates, identifies the most frequently missed questions, and classifies question performance as Easy, Moderate, or Difficult.

2. Objectives

• Compare each student's answers with the correct answer key.

• Count correct and missed answers for every question.

• Calculate question-wise success rates as percentages.

• Identify the question or questions with the greatest number of missed answers.

• Classify questions by performance to help teachers identify difficult topics.

3. Assumptions

• All students answer the same set of questions.

• The answer key contains one correct answer for each question.

• Student answers are represented as strings and compared question by question.

• Comparison ignores letter case and surrounding spaces.

• Easy: success rate ≥ 70%; Moderate: 40% ≤ success rate < 70%; Difficult: success rate < 40%.

• At least one student's answers are provided, and every answer row has the same length as the key.

4. Algorithm

1. Start.

2. Read or define the answer key and the answers submitted by all students.

3. Count correct answers for each question by comparing student responses with the key.

4. Calculate missed answers = total students – correct answers.

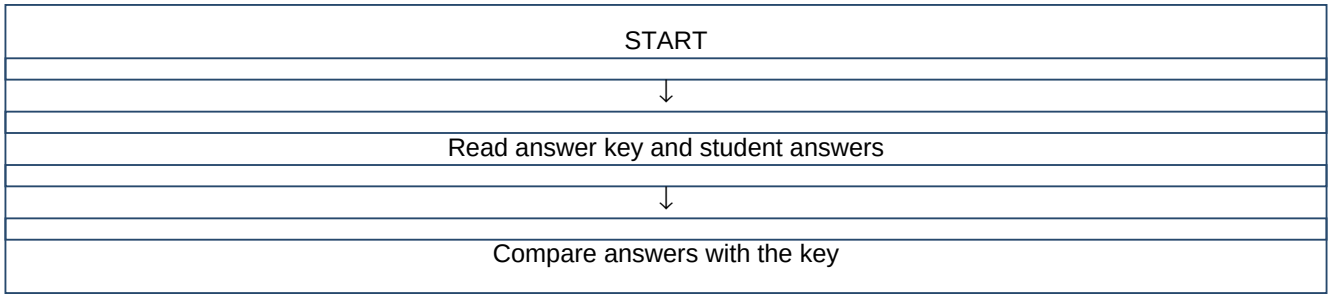
5. Calculate success rate = (correct answers ÷ total students) × 100.

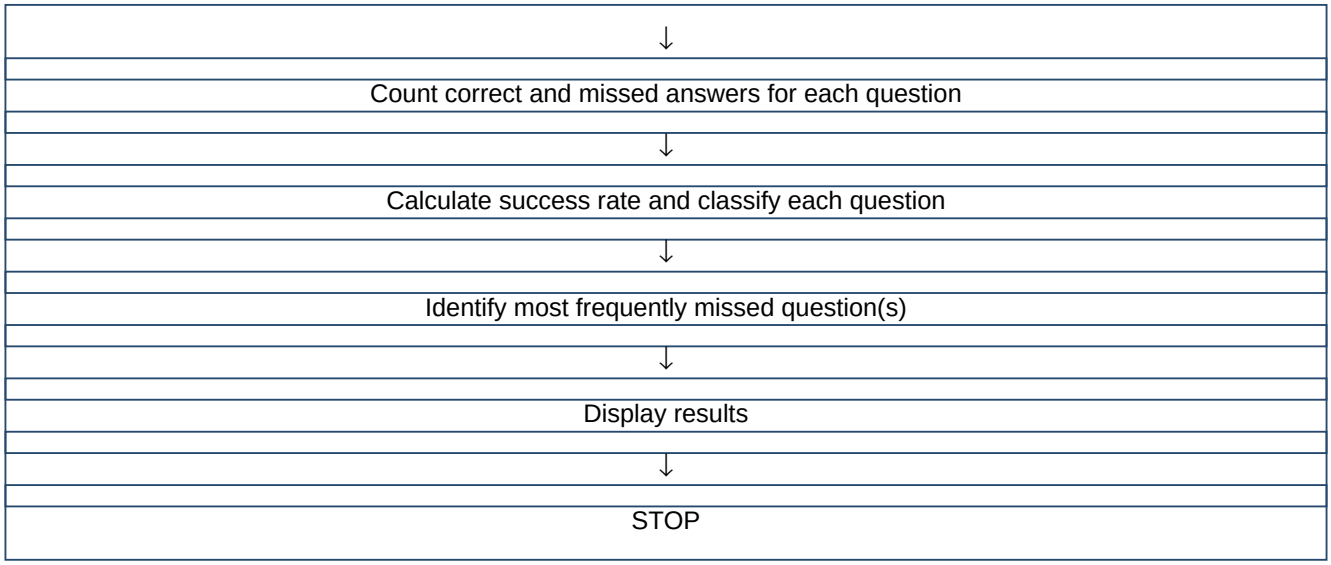
6. Classify the question: Easy (≥70%), Moderate (≥40% and <70%), otherwise Difficult.

7. Find the maximum missed-answer count and list every question that has that count.

8. Display the question-wise analysis and stop.

5. Flowchart




↓
Count correct and missed answers for each question
↓
Calculate success rate and classify each question
↓
Identify most frequently missed question(s)
↓
Display results
↓
STOP

6. Pseudocode

```
BEGIN
    INPUT answer_key
    INPUT student_answers
    total_students = number of student answer rows
    IF total_students == 0:
        REPORT error and stop

    FOR each question position i:
        correct_count = 0
        FOR each student's answer row:
            IF answer[i] matches answer_key[i] (case-insensitive):
                correct_count = correct_count + 1

        missed_count = total_students - correct_count
        success_rate = correct_count / total_students * 100

        IF success_rate >= 70:
            classification = "Easy"
        ELSE IF success_rate >= 40:
            classification = "Moderate"
        ELSE:
            classification = "Difficult"

        DISPLAY question, correct_count, missed_count,
                success_rate, classification

    FIND maximum missed_count
    DISPLAY all questions with that maximum
END
```

7. Python Source Code

Save the following program as **exam_difficulty_analysis.py**. The same source file is also provided separately with this report.

```
# Examination Question Difficulty and Performance Analysis

def analyze_exam(answer_key, student_answers):
    total_students = len(student_answers)
    total_questions = len(answer_key)

    if total_students == 0:
        raise ValueError("At least one student's answers are required.")

    correct_counts = [0] * total_questions

    for answers in student_answers:
        if len(answers) != total_questions:
            raise ValueError("Every student must answer the same number of questions.")

        for i in range(total_questions):
            if answers[i].strip().upper() == answer_key[i].strip().upper():
                correct_counts[i] += 1

    results = []
    for i, correct in enumerate(correct_counts):
        missed = total_students - correct
        success_rate = (correct / total_students) * 100

        if success_rate >= 70:
            classification = "Easy"
        elif success_rate >= 40:
            classification = "Moderate"
        else:
            classification = "Difficult"

        results.append({
            "question": i + 1,
            "correct": correct,
            "missed": missed,
            "success_rate": success_rate,
            "classification": classification
        })

    max_missed = max(item["missed"] for item in results)
    most_missed = [
        item["question"] for item in results
        if item["missed"] == max_missed
    ]

    return results, max_missed, most_missed
```

```
def main():
    answer_key = ["A", "B", "C", "D", "A"]
    student_answers = [
        ["A", "B", "C", "A", "A"],
        ["A", "C", "C", "D", "B"],
        ["B", "B", "A", "D", "A"],
        ["A", "B", "C", "C", "A"],
        ["A", "A", "C", "D", "B"]
    ]

    results, max_missed, most_missed = analyze_exam(
        answer_key, student_answers
    )

    print("=" * 58)
    print("EXAMINATION QUESTION DIFFICULTY")
    print("AND PERFORMANCE ANALYSIS")
    print("=" * 58)
    print(f"Total Students: {len(student_answers)}")
    print(f"Total Questions: {len(answer_key)}")
    print("\nQUESTION-WISE ANALYSIS")
    print("-" * 58)
    print(f"{'Question':<12}{'Correct':<10}{'Missed':<10}"
          f"{'Success Rate':<16}{'Classification'}")
    print("-" * 58)

    for item in results:
        print(
            f"Q{item['question']:<11}"
            f"{item['correct']:<10}"
            f"{item['missed']:<10}"
            f"{item['success_rate']:.0f}%{'':<12}"
            f"{item['classification']}"
        )

    print("\nMost Frequently Missed Questions:",
          ", ".join(f"Q{q}" for q in most_missed))
    print(f"Maximum Missed Answers: {max_missed}")
    print("\nAnalysis Completed Successfully.")

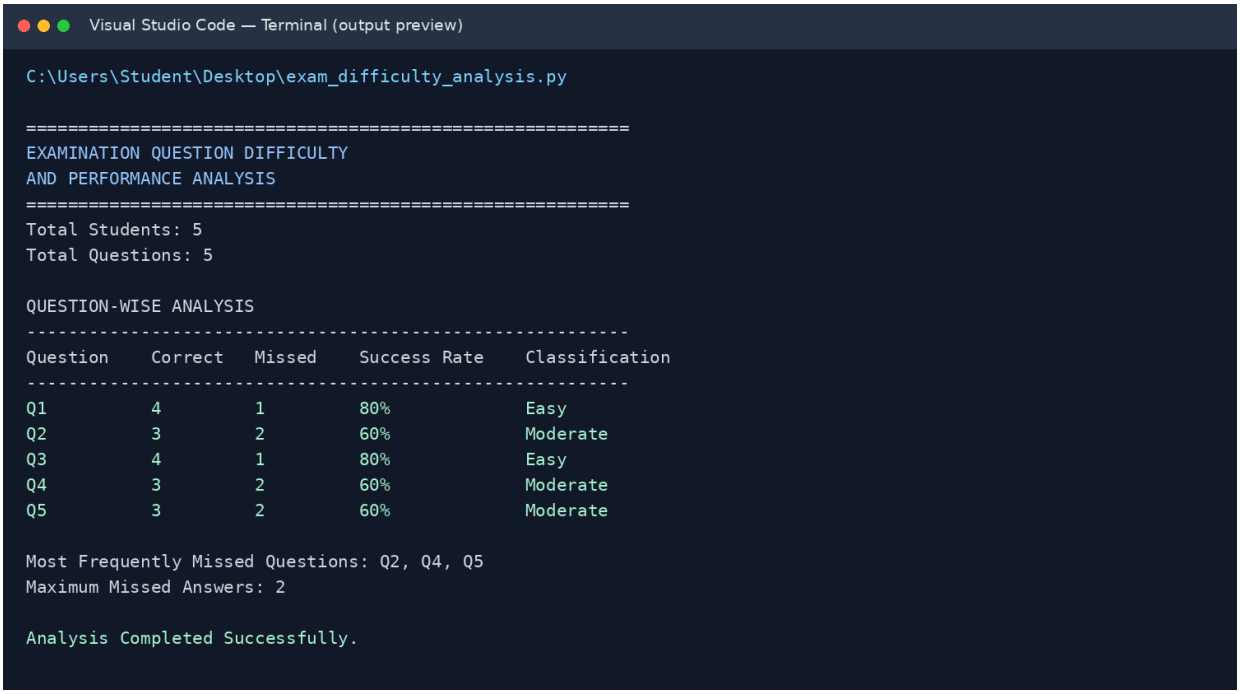

if __name__ == "__main__":
    main()
```

8. Test Cases

Test	Input / Scenario	Expected result
1. Sample data	Key: A B C D A; 5 student rows	Q1=80% Easy; Q2=60% Moderate; Q3=80% Easy; Q4=60% Moderate; Q5=60% Mo
2. All correct	Every student's answers match the key	Every question has 100% success and is Easy.
3. All incorrect	No student's answer matches any key entry	Every question has 0% success and is Difficult; all questions tie for most missed.
4. Case-insensitive	Key uses A; response uses a	The answer is counted as correct.
5. No students	Empty student answer list	Program reports that at least one student is required.

9. Output Screenshot

The image below is a generated terminal-style preview of the actual Python program's output. It is not a direct capture from the Visual Studio Code application. To obtain a genuine VS Code screenshot, open the supplied .py file in VS Code, run it in the integrated terminal, and take a screenshot.



10. Conclusion

The program compares student responses with an answer key and summarizes the performance of each question. It computes correct and missed answer counts, calculates success rates, identifies the most frequently missed questions, and labels questions Easy, Moderate, or Difficult. The results can help instructors identify topics that may need additional explanation and improve future teaching and assessment.